HashiCorp Vault Security Benchmark

Enterprise Edition

46 Controls Across 10 Domains, with Reference Architectures

Prepared for Customer Distribution

Table of Contents

Table of Contents 1
Overview 2
Platform Hardening 3
1. Enable TLS Everywhere 4
2. Use Integrated Storage (Raft) 4
3. Enable Audit Logging 5
4. Auto-Unseal with Cloud KMS 6
5. Restrict Root Token Usage 6
Auth & Identity 7
6. OIDC Authentication 8
7. MFA Enforcement 8
8. Short TTL Tokens 9
9. Least Privilege Policies 9
10. Identity Group Mapping 10
Secrets Engines 11
11. Use KV Version 2 12
12. Dynamic Secrets 12
13. Versioning Limits 13
14. Path Segmentation 13
15. Response Wrapping 14
PKI 15
16. Short-lived Certificates 16
17. Intermediate CAs 16
18. CRL and OCSP 17
19. Secure Root CA 18
Transit 18
20. Disable Key Export 19
21. Rotate Keys 19
22. Derived Keys 20
23. Scoped Access (Transit) 21
Kubernetes 21
24. Kubernetes Auth Method 22
25. RBAC Restrictions 22
26. Network Policies 23
27. Pod Security 24

28. Secrets Injection 25
AWS 26
29. KMS Auto-Unseal 27
30. IAM Auth 27
31. AWS PrivateLink 28
32. Security Groups 29
33. IMDSv2 Enforcement 29
Azure 30
34. Key Vault Auto-Unseal 31
35. Managed Identity 31
36. Azure Private Link 32
37. NSGs 33
38. AAD Integration 34
Monitoring 34
39. Enable Metrics 35
40. Central Logging 35
41. SIEM Integration 36
42. Alerting Rules 37
DR 37
43. Encrypted Backups 38
44. Snapshot Protection 38
45. Restore Testing 39
46. Unseal and Recovery Key Security 40
Reference Architectures 41
Vault HA (Raft) 42
Vault + EKS 42
Vault + AKS 42
Network Segmentation 42

Overview

This benchmark provides 46 security controls for HashiCorp Vault deployments, organized into ten domains: Platform Hardening, Authentication & Identity, Secrets Engines, PKI, Transit, Kubernetes, AWS, Azure, Monitoring, and Disaster Recovery. Each control includes a description, rationale, potential operational impact, audit procedures, remediation steps, and cloud-specific notes for AWS and Azure where applicable. Controls that are not specific to a cloud platform are marked accordingly.

Platform Hardening

1. Enable TLS Everywhere

Description: Ensure all Vault listener and cluster communications (client API, replication, Raft) use TLS with valid certificates, and that plaintext HTTP is disabled.

Rationale: Vault transmits secrets, tokens, and unseal material over the network. Without TLS, this traffic can be intercepted or tampered with via man-in-the-middle attacks, exposing credentials and encryption keys.

Impact: Requires certificate issuance and rotation processes (ideally via Vault's own PKI engine or an external CA) and may add latency for TLS handshakes; misconfigured certificates can cause client connection failures.

Audit

- Run vault read sys/config/state/sanitized and confirm the listener stanza shows tls_disable = false
- Inspect the Vault server configuration file for the listener "tcp" stanza and confirm tls_cert_file / tls_key_file are set and tls_disable is absent or false
- Run openssl s_client -connect <vault-addr>:8200 to verify the certificate chain and negotiated TLS version
- Confirm tls_min_version is set to "tls12" or higher

Remediation

- Set tls_disable = false (or remove the parameter) and supply valid tls_cert_file / tls_key_file values
- Use Vault's PKI secrets engine or an enterprise CA to issue and rotate Vault server certificates
- Enforce tls_min_version = "tls12" and disable weak cipher suites
- Enable TLS for cluster/replication ports as well as the client API port

Cloud Notes

- AWS: Terminate TLS at Vault itself, not only at the load balancer, to maintain end-to-end encryption. ACM-issued certificates on an ALB do not replace TLS on the Vault listener.
- Azure: Store TLS private keys in Azure Key Vault for Vault to reference and enable TLS on Application Gateway / Load Balancer in addition to the Vault listener TLS configuration.

2. Use Integrated Storage (Raft)

Description: Use Vault's Integrated Storage (Raft) backend for HA deployments instead of an external storage backend such as Consul, providing built-in consensus and simplified operations.

Rationale: Raft storage reduces external dependencies and attack surface, simplifies backup and restore via snapshots, and provides strong consistency guarantees for cluster state.

Impact: Requires careful planning of cluster size (an odd number of nodes, typically 3 or 5) and low-latency network connectivity between nodes on the Raft port (default 8201); migrating from another backend requires a formal migration process.

Audit

- Run vault status and confirm Storage Type: raft
- Run vault operator raft list-peers and confirm the expected number of voters and that all nodes report a healthy state
- Run vault operator raft configuration and review cluster topology

Remediation

- Configure storage "raft" { path = "...", node_id = "..." } in the server configuration
- Deploy an odd number of voting nodes (3 or 5) distributed across availability zones for quorum resilience
- Restrict the Raft port (default 8201) to cluster members only

Cloud Notes

- AWS: Distribute Raft nodes across multiple Availability Zones with low-latency private networking (VPC peering or Transit Gateway as needed).
- Azure: Distribute Raft nodes across Availability Zones within a region and ensure NSGs permit inter-node Raft traffic on port 8201.

3. Enable Audit Logging

Description: Enable at least one (preferably two) audit devices so that all requests and responses to Vault are logged for forensic and compliance purposes.

Rationale: Audit logs are the primary mechanism for detecting unauthorized access, investigating incidents, and meeting compliance requirements such as PCI-DSS and SOC 2. When multiple audit devices are configured, Vault requires that at least one device successfully persists each log entry before the response is returned; it blocks requests only when it cannot successfully write to any enabled audit device. Configuring a second audit device therefore provides both resilience against a single device failure and an additional layer of assurance that log writes succeed.

Impact: Audit logs grow quickly and contain hashed but potentially sensitive data, requiring log rotation, secure storage, and a forwarding pipeline. If all configured audit devices fail simultaneously, Vault becomes unavailable. Note that an audit device that hangs rather than failing fast can stall Vault even when another device is healthy, so device health monitoring is essential.

Audit

- Run vault audit list -detailed and confirm at least one audit device is enabled and active
- Review the audit log file for recent entries and confirm rotation and retention are configured

Remediation

- Enable a file audit device: vault audit enable file file_path=/var/log/vault/audit.log
- Enable a second audit device (for example syslog, or a separate file path) for redundancy
- Configure log rotation (for example logrotate) and forward logs to a central log store or SIEM
- Monitor audit device health proactively — a hung or slow device can stall Vault even when another device is present.

Cloud Notes

- AWS: Forward audit logs to CloudWatch Logs or to S3 with Object Lock enabled for immutability, via a log shipping agent.
- Azure: Forward audit logs to an Azure Monitor Log Analytics workspace via the Azure Monitor Agent.

4. Auto-Unseal with Cloud KMS

Description: Configure Vault to automatically unseal using a cloud key management service (AWS KMS, Azure Key Vault, or GCP Cloud KMS) instead of manual entry of Shamir key shares.

Rationale: Manual unseal requires gathering key holders after every restart, which is operationally fragile and can cause extended outages. Auto-unseal via a cloud KMS removes this dependency while shifting protection of the master key to a managed, HSM-backed service.

Impact: Creates a dependency on the cloud KMS being available; the IAM role or identity granted to Vault for the KMS key becomes a high-value target and must be tightly scoped. Recovery keys (rather than unseal keys) are then used for operations such as generate-root.

Audit

- Run vault status and confirm Seal Type shows awskms, azurekeyvault, or an equivalent cloud seal (not shamir)
- Review the server configuration for a seal "awskms" { ... } or seal "azurekeyvault" { ... } stanza
- Confirm recovery keys, not unseal keys, have been generated and distributed for operations such as generate-root and rekey

Remediation

- Add the appropriate seal stanza to the Vault configuration referencing the cloud KMS key
- Restrict the IAM policy or RBAC role used by Vault to only the encrypt, decrypt, and describe-key permissions on the specific key
- Securely distribute and store recovery key shares per control 46

Cloud Notes

- AWS: See control 29 for AWS KMS-specific guidance, including key policy scoping and key rotation.
- Azure: See control 34 for Azure Key Vault-specific guidance, including soft-delete and purge protection.

5. Restrict Root Token Usage

Description: Eliminate routine use of the initial root token. Revoke or tightly time-bound any root tokens and require a documented break-glass procedure for emergency root access.

Rationale: The root token bypasses all policy checks. If compromised or used routinely, it provides unrestricted access to all secrets and configuration, undermining the value of least-privilege policies defined elsewhere.

Impact: Requires establishing a break-glass process (for example, generate-root using recovery or unseal key holders) for emergency access, and may slow certain administrative tasks that previously relied on the root token.

Audit

- Attempt vault token lookup against the original setup root token and confirm it returns an error indicating the token has been revoked
- Search audit logs for any requests where policies include "root" outside of initial setup
- Review active tokens with elevated policies for any that were issued with the root policy

Remediation

- Run vault token revoke <root-token> immediately after initial cluster setup is complete
- Use vault operator generate-root with recovery or unseal key holders only when emergency root access is genuinely required, and revoke the generated token immediately after use
- Create scoped administrative policies for day-to-day administrative tasks instead of relying on the root token

Cloud Notes

- AWS: Store recovery key shares used for generate-root in separate, access-logged locations such as distinct AWS accounts, with restricted IAM policies and CloudTrail logging on access.
- Azure: Store recovery key shares used for generate-root in separate Key Vaults or offline secure storage, with access logged via Azure Monitor.

Auth & Identity

6. OIDC Authentication

Description: Configure the OIDC auth method so users authenticate to Vault using an existing identity provider (such as Okta, Azure AD, or Google Workspace) rather than static credentials.

Rationale: Centralizing authentication through an enterprise identity provider enables consistent enforcement of password policies and conditional access, and allows immediate de-provisioning of access when an employee leaves the organization.

Impact: Requires the identity provider to be reachable from Vault and from end-user browsers or CLIs. Misconfigured redirect URIs or claim mappings can lock users out of Vault entirely.

Audit

- Run vault auth list and confirm an oidc/ mount is enabled
- Run vault read auth/oidc/config and verify oidc_discovery_url, default_role, and allowed redirect URIs are correctly scoped
- Run vault list auth/oidc/role and vault read auth/oidc/role/<role> for each role and confirm role-to-policy bindings are least-privilege

Remediation

- Enable the OIDC auth method: vault auth enable oidc
- Configure the discovery URL and client credentials: vault write auth/oidc/config oidc_discovery_url=<url> oidc_client_id=<id> oidc_client_secret=<secret>
- Define roles that map identity provider groups or claims to specific Vault policies, and avoid mapping any group to broad administrative policies

Cloud Notes

- AWS: Where applicable, integrate with AWS IAM Identity Center as the OIDC provider, or with the organization's existing identity provider federated to AWS.
- Azure: Integrate with Azure AD (Microsoft Entra ID) as the OIDC issuer, using a dedicated App Registration scoped to Vault.

7. MFA Enforcement

Description: Require multi-factor authentication for Vault logins or for sensitive operations using Vault's Login MFA capability (TOTP, Duo, Okta, or PingID).

Rationale: A compromised password or token alone should not be sufficient to access Vault, particularly for privileged accounts. MFA significantly raises the bar against credential-based attacks.

Impact: Adds an additional step to every login and requires users to enroll an MFA method. A documented recovery procedure for lost MFA devices is required to avoid lockouts.

Audit

- Run vault read sys/mfa/method/<type>/<name> for each configured MFA method and confirm it is correctly configured
- Run vault list identity/mfa/login-enforcement and vault read identity/mfa/login-enforcement/<name> and confirm enforcement is bound to the relevant auth methods, groups, or all logins

Remediation

- Configure an MFA method, for example vault write sys/mfa/method/duo/<name> ... or a TOTP method
- Create a login enforcement configuration binding the MFA method to the relevant auth mounts or identity groups: vault write identity/mfa/login-enforcement/<name> mfa_method_ids=<id> auth_method_accessors=<accessor>
- Document and test an MFA reset or break-glass procedure for users who lose their MFA device

Cloud Notes

- AWS: Vault Login MFA is independent of any MFA configured for AWS IAM users; integrate with Duo, Okta, or PingID as the Vault-side MFA provider rather than relying on AWS-side MFA alone.
- Azure: Vault Login MFA is complementary to, not a substitute for, Azure AD conditional access policies; both should be configured.

8. Short TTL Tokens

Description: Configure conservative default and maximum TTLs for tokens and leases so that credentials issued by Vault automatically expire and require renewal or re-authentication.

Rationale: Long-lived tokens that are leaked or forgotten remain a standing risk indefinitely. Short TTLs limit the impact of a compromised token and force periodic re-validation of identity.

Impact: Applications must implement token renewal logic or re-authentication. TTLs that are too short relative to an application's workflow can cause service disruptions if renewal is not handled correctly.

Audit

- Run vault read sys/auth/<mount>/tune for each enabled auth method and review default_lease_ttl and max_lease_ttl
- Run vault read sys/mounts/<mount>/tune for each secrets engine mount and review configured TTLs
- Spot-check issued tokens with vault token lookup <token> and review the ttl and explicit_max_ttl fields

Remediation

- Tune auth method TTLs to organizational policy, for example: vault auth tune -default-lease-ttl=1h -max-lease-ttl=4h <mount>
- Set conservative system-wide default and maximum lease TTLs in the server configuration as a backstop
- Use periodic tokens with renewal only where long-running processes genuinely require it, paired with Vault Agent for automatic renewal

Cloud Notes

- AWS: Align Vault token TTLs with the TTLs of temporary credentials issued via the AWS secrets engine and AWS STS.
- Azure: Align Vault token TTLs with the lifetime of credentials issued via the Azure secrets engine and Managed Identity-based roles.

9. Least Privilege Policies

Description: Author Vault ACL policies that grant only the specific paths and capabilities (read, create, update, delete, list) required for each role, avoiding wildcard or overly broad grants.

Rationale: Overly permissive policies, such as granting full capabilities on secret/* or any/*, effectively provide near-root access to entire secret trees and undermine segmentation between teams and applications.

Impact: Requires upfront design effort to map applications and teams to specific paths. Overly restrictive policies can break applications and require iterative tuning.

Audit

- Run vault policy list to inventory all policies
- Run vault policy read <policy> for each policy and review for wildcard paths combined with broad capability sets, particularly the sudo capability
- Cross-reference policies attached to roles via vault read auth/<mount>/role/<role> to confirm scoping is appropriate for each application or team

Remediation

- Rewrite policies to scope paths to specific mounts or prefixes per application or team, for example path "secret/data/team-a/*"
- Remove the sudo capability from non-administrative policies
- Use templated policies that reference identity metadata (for example {{identity.entity.aliases...}}) to scope access dynamically based on identity

Cloud Notes

- AWS: Apply the same least-privilege principle used for IAM policies; avoid the Vault equivalent of an IAM statement granting Action: * and Resource: *.
- Azure: Align Vault policy scoping with Azure RBAC role assignments so that access boundaries are consistent across the stack.

10. Identity Group Mapping

Description: Use Vault's Identity secrets engine to map external groups from LDAP, OIDC, Okta, or Azure AD to internal Vault groups, and attach policies to those groups rather than to individual entities.

Rationale: Group-based policy assignment scales better and reduces drift compared with assigning policies to individual users or tokens, and ensures access changes automatically when a user's group membership changes in the identity provider.

Impact: Requires accurate and timely group synchronization from the identity provider. Misconfigured group mappings can result in over- or under-granted access.

Audit

- Run vault list identity/group/name to inventory internal groups
- Run vault read identity/group/name/<group> for each group and confirm the policies and member references are correct
- Run vault list identity/group-alias and confirm external groups are correctly aliased to the corresponding internal groups

Remediation

- Create internal groups: vault write identity/group name=<name> policies=<policy>
- Create group aliases linking external identity provider groups to internal Vault groups: vault write identity/group-alias name=<external-group> mount_accessor=<accessor> canonical_id=<internal-group-id>
- Periodically audit group membership in Vault against the source identity provider to detect drift

Cloud Notes

- AWS: Map AWS IAM Identity Center groups, or identity-provider groups federated through it, to corresponding Vault identity groups.
- Azure: Map Azure AD security groups directly to Vault identity groups using the group claims returned by the OIDC or Azure auth method.

Secrets Engines

11. Use KV Version 2

Description: Use the Key/Value Version 2 secrets engine, with versioning, soft-delete, and check-and-set support, instead of Version 1 for static secrets.

Rationale: KV v2 retains version history, supports soft-delete, undelete, and destroy operations, and supports check-and-set to prevent accidental overwrites, all of which improve operational safety and recoverability compared with KV v1's single-version, overwrite-only model.

Impact: KV v2 paths differ from v1 (secret/data/<path> and secret/metadata/<path> versus secret/<path>), which requires updating application integrations and policies when migrating from v1.

Audit

- Run vault secrets list -detailed and confirm relevant KV mounts show version 2 under Options
- Run vault read secret/metadata/<path> for a sample of secrets and confirm version metadata is being tracked

Remediation

- For new mounts, enable KV v2 directly: vault secrets enable -version=2 -path=secret kv
- For existing v1 mounts, either upgrade in place where supported (vault kv enable-versioning secret/) or migrate to a new v2 mount and update application paths and policies accordingly

Cloud Notes

- Not cloud-specific: applies equally regardless of the underlying infrastructure platform.

12. Dynamic Secrets

Description: Where supported, use dynamic secrets engines (database, AWS, Azure, and similar) that generate short-lived, on-demand credentials instead of storing static, long-lived credentials in KV.

Rationale: Dynamic secrets are unique per lease, expire automatically, and are automatically revoked, eliminating the risk of long-lived shared credentials being copied, leaked, or reused after access is no longer required.

Impact: Requires Vault to hold privileged credentials capable of creating and revoking downstream credentials, for example a database administrator account. This credential itself becomes a high-value target requiring strict access controls. Some legacy systems may not tolerate frequent credential rotation.

Audit

- Run vault secrets list and identify which engines are static (KV) versus dynamic (database, aws, azure, and similar)
- For each dynamic engine, run vault list <mount>/roles and vault read <mount>/roles/<role> and confirm TTLs and downstream permissions are least-privilege
- Review active lease counts and durations via vault list sys/leases/lookup/<mount>

Remediation

- Enable applicable dynamic secrets engines, for example vault secrets enable database, vault secrets enable aws, or vault secrets enable azure
- Configure roles with minimal downstream privileges and TTLs aligned with control 8
- Migrate static, long-lived credentials currently stored in KV to dynamic equivalents where feasible

Cloud Notes

- AWS: Use the AWS secrets engine to generate short-lived IAM credentials or STS tokens scoped to specific IAM policies, instead of storing static IAM access keys in Vault.
- Azure: Use the Azure secrets engine to generate short-lived Service Principal credentials scoped to specific Azure roles and resource groups.

13. Versioning Limits

Description: Configure the max_versions setting on KV v2 mounts to bound how many historical versions of a secret are retained.

Rationale: Without a configured limit, KV v2 retains historical versions of every secret, increasing storage growth and extending the window during which old, potentially compromised secret values remain recoverable through version history.

Impact: Setting max_versions too low may remove historical versions needed for rollback or audit before they have been reviewed. Setting it too high increases storage usage and retention of stale secret values.

Audit

- Run vault read secret/config (or <mount>/config for the relevant KV v2 mount) and confirm max_versions is set to an organization-approved value rather than left at the default
- Run vault read secret/metadata/<path> for a sample of secrets and compare the number of retained versions against the configured max_versions

Remediation

- Set an organization-approved version limit, for example: vault write secret/config max_versions=10
- For highly sensitive secrets, consider a lower max_versions combined with delete_version_after to enforce automatic expiry of old versions

Cloud Notes

- Not cloud-specific: applies equally regardless of the underlying infrastructure platform.

14. Path Segmentation

Description: Structure secret paths and mounts so that different teams, environments (dev, staging, production), and applications occupy clearly separated namespaces, for example secret/data/<environment>/<team>/<application>/...

Rationale: Clear path segmentation makes it possible to write simple, least-privilege policies as described in control 9, and reduces the risk of cross-team or cross-environment secret access caused by overly broad path patterns.

Impact: Retrofitting path segmentation into an existing flat secret structure requires migrating secrets and updating application configuration and policies, which can be disruptive if not carefully sequenced.

Audit

- Run vault kv list secret/ recursively and review the existing path hierarchy for a consistent, predictable structure
- Cross-reference the path hierarchy against vault policy read <policy> output to confirm policies align with the path structure rather than relying on broad top-level wildcards

Remediation

- Define and document a standard path naming convention, for example <environment>/<team>/<application>/<secret-name>
- Use separate KV mounts per environment (for example secret-dev/ and secret-prod/) where strong isolation of audit trails and access boundaries is required
- Migrate existing secrets to the new structure and update policies and application configuration accordingly

Cloud Notes

- AWS: Align path segmentation with AWS account or organizational unit boundaries, for example separate AWS accounts per environment mapping to separate Vault path prefixes.
- Azure: Align path segmentation with Azure subscription or resource group boundaries.

15. Response Wrapping

Description: Use Vault's response wrapping (wrap-ttl) feature when delivering sensitive initial values, such as initial passwords, recovery keys, or bootstrap tokens, to clients via a single-use wrapping token.

Rationale: Response wrapping ensures that only the intended recipient can unwrap and view the secret, and that any interception attempt is detectable: if a wrapped response is unwrapped by an unauthorized party, the legitimate recipient's subsequent unwrap attempt will fail.

Impact: Adds an additional unwrap step to client workflows. Wrapped tokens have their own short TTL, and if not unwrapped promptly the underlying data is permanently lost.

Audit

- Review onboarding and bootstrap automation, such as CI/CD pipelines and provisioning scripts, to confirm sensitive initial credentials are transmitted using wrap-ttl rather than in plaintext
- Review audit logs for unwrap operations and confirm wrapped responses are unwrapped exactly once and promptly after issuance

Remediation

- Add -wrap-ttl=<duration> to CLI or API calls that return sensitive initial values, for example vault login -wrap-ttl=60s, or set the equivalent X-Vault-Wrap-TTL header
- Update provisioning tooling, including Terraform and configuration management, to retrieve and immediately unwrap responses, and ensure the wrapping token itself is never persisted

Cloud Notes

- AWS: Use response wrapping when delivering initial Vault tokens to EC2 instances or ECS tasks during bootstrap, in conjunction with the AWS auth method.
- Azure: Use response wrapping when delivering initial Vault tokens to virtual machines or AKS nodes during bootstrap, in conjunction with the Azure auth method.

PKI

16. Short-lived Certificates

Description: Configure PKI roles to issue certificates with short TTLs, measured in hours to days rather than months or years, leveraging Vault PKI's ability to issue certificates on demand.

Rationale: Short-lived certificates significantly reduce the impact of a private key compromise, since a stolen key is useful for a much shorter window, and reduce reliance on revocation infrastructure because most certificates expire before revocation would otherwise matter.

Impact: Requires clients and services to automate certificate renewal, for example via Vault Agent, consul-template, or cert-manager. Applications that cache certificates without automated renewal will experience outages when those certificates expire.

Audit

- Run vault read pki/roles/<role> for each role and review ttl and max_ttl values to confirm alignment with policy, commonly 72 hours or less for workload certificates
- Run vault list pki/certs and inspect a sample with vault read pki/cert/<serial> to confirm issued certificate lifetimes match the role configuration

Remediation

- Set conservative TTLs on PKI roles, for example vault write pki/roles/<role> ttl=24h max_ttl=72h ...
- Deploy automated renewal using Vault Agent templates, consul-template, or cert-manager for Kubernetes workloads, so certificates are reissued well before expiry

Cloud Notes

- AWS: If AWS Certificate Manager Private CA is also in use, ensure the relationship between it and Vault PKI (for example as a subordinate) is clearly documented; otherwise keep internal service-to-service TLS entirely within Vault PKI.
- Azure: Ensure short-lived certificates issued by Vault PKI are trusted by Azure-hosted services, such as AKS ingress controllers, via correct distribution of the trust bundle.

17. Intermediate CAs

Description: Use a multi-tier PKI hierarchy in which Vault hosts one or more Intermediate Certificate Authorities signed by a Root CA, rather than issuing leaf certificates directly from a Root CA.

Rationale: Keeping the Root CA offline, or in a separate and more tightly controlled mount or namespace, and using Intermediate CAs for day-to-day issuance limits the impact if an Intermediate CA's key is compromised: it can be revoked and reissued without invalidating the entire trust chain.

Impact: Adds operational complexity in managing the relationship between root and intermediate mounts and in renewing intermediate certificates periodically. Clients must trust the full chain, including both root and intermediate, not just the leaf issuer.

Audit

- Run vault read pki/cert/ca_chain on the issuing mount and confirm the chain includes an intermediate CA rather than a self-signed root used directly for leaf issuance

- Run vault list pki/issuers (Vault 1.11 and later) and review the issuer hierarchy to confirm the root CA is either external or hosted in a separate, more restricted mount or namespace

Remediation

- Generate a root CA in a dedicated, tightly restricted mount: vault write pki/root/generate/internal ...
- Generate an intermediate CSR in a separate mount, sign it with the root, and import the signed certificate into the intermediate mount
- Use the intermediate mount for all day-to-day leaf certificate issuance as described in control 16, and consider sealing or revoking access to the root mount between signing events

Cloud Notes

- Not cloud-specific: this multi-tier hierarchy applies regardless of the hosting platform.

18. CRL and OCSP

Description: Configure Certificate Revocation List generation and, where supported, an OCSP responder so relying parties can check the revocation status of certificates issued by Vault PKI.

Rationale: Even with short-lived certificates as described in control 16, revocation mechanisms provide a way to immediately invalidate a certificate before its natural expiry, for example in response to a confirmed key compromise.

Impact: CRLs must be distributed to and checked by relying parties, requiring correct cdp_url configuration and client support. OCSP responders add an additional service dependency that relying parties query during the TLS handshake, which can add latency if not cached appropriately.

Audit

- Run vault read pki/config/crl and confirm CRL generation is enabled (disable=false) with an appropriate expiry value
- Run vault read pki/config/urls and confirm issuing_certificates and crl_distribution_points (and ocsp_servers if applicable) are populated with reachable URLs
- Where OCSP is enabled, query the pki/ocsp endpoint for a known certificate and confirm the responder returns a valid status

Remediation

- Enable CRL generation with an appropriate expiry, for example vault write pki/config/crl expiry=72h disable=false
- Configure issuing certificate and CRL distribution point URLs: vault write pki/config/urls issuing_certificates="<vault-addr>/v1/pki/ca" crl_distribution_points="<vault-addr>/v1/pki/crl"
- Where supported (Vault 1.13 and later), enable the PKI mount's OCSP responder and configure relying applications to perform OCSP checks where supported

Cloud Notes

- AWS: Host CRL and OCSP endpoints behind a public or private Application Load Balancer depending on whether relying parties are internal or external to the VPC.

- Azure: Host CRL and OCSP endpoints behind Application Gateway, ensuring DNS and routing allow all relying services to reach these endpoints.

19. Secure Root CA

Description: Protect the Root CA, whether hosted in Vault or externally, with maximum restrictions, including minimal use, restricted access policies, and offline or sealed storage when not actively signing intermediates.

Rationale: Compromise of the Root CA's private key invalidates the entire PKI trust hierarchy and requires reissuing every certificate and redistributing new trust anchors to every relying party, an extremely costly and disruptive event. The root must therefore be the most tightly protected component of the PKI.

Impact: Operational processes for signing new intermediates, such as periodic intermediate renewal, become deliberately slower and more manual, requiring a documented ceremony with multiple approvers.

Audit

- Run vault policy read <root-ca-policy> and confirm access to the root CA mount is restricted to a small, named set of administrators with no broad or wildcard policies
- If Vault Enterprise namespaces are in use, confirm the root CA mount resides in a separate, isolated namespace
- Review audit logs for any access to the root CA mount outside of scheduled intermediate-signing events

Remediation

- Restrict the root CA mount's policy to a dedicated PKI administrator group only, and require MFA per control 7
- After generating or signing intermediates, seal the Vault instance hosting the root CA, or revoke access tokens until the next signing event
- Document a formal, multi-person, logged root signing ceremony procedure

Cloud Notes

- AWS: Consider hosting the root CA in a separate, isolated AWS account with strict IAM boundaries from production workloads.
- Azure: Consider hosting the root CA in a separate subscription or management group with restricted Azure AD access.

Transit

20. Disable Key Export

Description: Ensure Transit secrets engine encryption keys are configured with exportable=false, which is the default, unless there is a documented and approved business need for key export.

Rationale: The Transit engine is designed to perform encryption as a service without ever exposing raw key material to clients. Allowing export defeats this core security property and creates a path for key material to leave Vault's control entirely.

Impact: If exportable=true is required for a legitimate use case such as key migration, that key's policy must be even more tightly restricted, and export operations should be specifically monitored.

Audit

- Run vault read transit/keys/<name> for each production key and confirm exportable: false.
- Review policies for any grants of the export capability on transit/export/* paths.
- Search audit logs for any historical use of transit/export/<key> endpoints.

Remediation

- For new keys, do not set exportable=true; the default is false: vault write -f transit/keys/<name>
- For any existing keys with exportable=true that do not require it: note that once exportable is enabled on a key it cannot be disabled — this is a one-way latch. The correct remediation is to retire such keys by creating a new non-exportable key, rewrapping data encrypted under the old key, and deactivating the old key per organizational key-rotation procedures.
- If export is genuinely required, restrict the export capability to a single, MFA-protected policy.

Cloud Notes

- Not cloud-specific: applies to the Transit engine regardless of the underlying infrastructure platform.

21. Rotate Keys

Description: Establish a regular rotation schedule for Transit and other encryption keys using Vault's built-in rotate operation, and configure min_decryption_version to phase out old key versions over time.

Rationale: Periodic key rotation limits the amount of data encrypted under any single key version and reduces the impact if a key version is ever compromised. Vault's Transit engine supports rotation without immediately re-encrypting existing data, through versioned keys.

Impact: Old ciphertext remains decryptable with prior key versions until min_decryption_version is increased. Increasing it without first rewrapping data will make older ciphertext permanently undecryptable, so rewrapping should precede any increase to min_decryption_version.

Audit

- Run vault read transit/keys/<name> and review latest_version, min_decryption_version, and min_encryption_version, and check rotation timestamps if auto_rotate_period is configured
- Confirm whether auto_rotate_period is set for automatic rotation, or whether rotation is performed manually on a documented schedule

Remediation

- Rotate immediately with vault write -f transit/keys/<name>/rotate, or configure automatic rotation: vault write transit/keys/<name>/config auto_rotate_period=<duration> (for example 90 days)
- Periodically rewrap existing ciphertext with the latest key version using vault write transit/rewrap/<name> ciphertext=<...>
- Once all data is confirmed rewrapped, increase min_decryption_version to retire old key versions

Cloud Notes

- AWS: If AWS KMS is used for auto-unseal (control 29), enable AWS KMS automatic annual key rotation as a complementary, independent control.
- Azure: If Azure Key Vault is used for auto-unseal (control 34), configure key rotation policies on the Key Vault key in addition to Transit key rotation.

22. Derived Keys

Description: For multi-tenant use cases, configure Transit keys with derived=true and use per-context key derivation so that each tenant or context receives a cryptographically distinct key derived from a single base key.

Rationale: Derived keys allow a single Transit key to securely serve many tenants without requiring a separate key per tenant, while still cryptographically isolating each tenant's data: one tenant cannot decrypt another tenant's data even with API access, provided contexts are not shared or guessable.

Impact: The context parameter must be supplied on every encrypt and decrypt operation and must be managed carefully; lost or inconsistent contexts make data permanently undecryptable. The derived setting cannot be changed after key creation.

Audit

- Run vault read transit/keys/<name> and confirm derived: true for keys intended for multi-tenant use
- Review application integration to confirm a unique, consistently derivable context value is used per tenant and is supplied on every encrypt and decrypt call

Remediation

- Create the key with derivation enabled: vault write -f transit/keys/<name> derived=true
- Ensure application code derives the context value deterministically per tenant, for example from a tenant identifier, and passes it on every encrypt and decrypt call
- If deterministic ciphertext for identical plaintext and context is required, evaluate convergent_encryption=true alongside derivation, considering the information-leakage tradeoffs for repeated values

Cloud Notes

- Not cloud-specific: applies to the Transit engine regardless of the underlying infrastructure platform.

23. Scoped Access (Transit)

Description: Restrict Transit policy grants so that each application or service can perform encrypt and decrypt operations only against its own specific named key or keys, not against all keys in the mount.

Rationale: Without scoping, any service with access to transit/encrypt/* or transit/decrypt/* could encrypt or decrypt data belonging to other services using their keys, breaking tenant or service isolation even where the underlying data store is otherwise segmented.

Impact: Requires maintaining a controlled mapping between applications and their named Transit keys, and updating policies whenever new keys or services are introduced.

Audit

- Run vault policy read <app-policy> and confirm transit/encrypt/<key> and transit/decrypt/<key> paths reference specific key names rather than wildcarded paths such as transit/encrypt/*
- Run vault list transit/keys and cross-reference the key inventory against the applications and policies that reference each key

Remediation

- Define one Transit key per application, service, or well-defined data domain: vault write -f transit/keys/<app-name>
- Write policies granting the update capability only on transit/encrypt/<app-name> and transit/decrypt/<app-name>, and the relevant transit/datakey/... paths if data keys are generated, without wildcarding across keys

Cloud Notes

- Not cloud-specific: applies to the Transit engine regardless of the underlying infrastructure platform.

Kubernetes

24. Kubernetes Auth Method

Description: Configure Vault's Kubernetes auth method so that pods can authenticate to Vault using their projected Kubernetes service account tokens, without static credentials.

Rationale: Kubernetes auth ties Vault access directly to a pod's Kubernetes identity, namely its service account and namespace, which Kubernetes automatically provisions and rotates, eliminating the need to distribute and manage static Vault tokens to workloads.

Impact: Requires Vault to have network access to the Kubernetes API server for token review, and requires careful role binding to avoid mapping a service account that is shared by many unrelated pods to a single Vault policy.

Audit

- Run vault auth list and confirm a kubernetes/ mount is enabled
- Run vault read auth/kubernetes/config and confirm kubernetes_host points to the correct API server and that kubernetes_ca_cert and the token reviewer JWT are correctly configured
- Run vault list auth/kubernetes/role and vault read auth/kubernetes/role/<role> for each role and confirm bound_service_account_names and bound_service_account_namespaces are scoped to specific service accounts and namespaces, not wildcards

Remediation

- Enable the Kubernetes auth method: vault auth enable kubernetes
- Configure the connection to the Kubernetes API: vault write auth/kubernetes/config kubernetes_host=https://<k8s-api>:443 kubernetes_ca_cert=@ca.crt token_reviewer_jwt=@token
- Define a role per application: vault write auth/kubernetes/role/<app> bound_service_account_names=<sa> bound_service_account_namespaces=<ns> policies=<policy> ttl=1h

Cloud Notes

- AWS: For EKS, ensure the token reviewer service account is bound to the system:auth-delegator ClusterRole and that Vault can reach the EKS API endpoint, whether public or private depending on cluster networking.
- Azure: For AKS, the same system:auth-delegator requirement applies; ensure Vault can reach the AKS API server endpoint, accounting for private cluster configurations.

25. RBAC Restrictions

Description: Apply Kubernetes RBAC so that Vault-related service accounts, including the token reviewer and the Vault Agent Injector, hold only the minimal ClusterRole or Role permissions required, and that application namespaces do not grant broad access to Vault-related secrets or configmaps.

Rationale: The Vault Kubernetes auth integration requires Vault to call the Kubernetes TokenReview API. If the service account used for this is over-permissioned, for example bound to cluster-admin, compromising it could grant broad cluster access well beyond Vault token review.

Impact: Requires careful, ongoing maintenance of RBAC bindings as the cluster and Vault integration evolve. Overly restrictive RBAC can break the auth method's ability to validate tokens.

Audit

- Run kubectl get clusterrolebinding -o wide and review bindings for the Vault token reviewer service account
- Run kubectl describe clusterrole system:auth-delegator and confirm only the intended service account is bound to it
- Run kubectl get rolebindings,clusterrolebindings -A -o json and search for any overly broad bindings, such as cluster-admin, involving Vault-related service accounts

Remediation

- Bind the Vault token reviewer service account only to the system:auth-delegator ClusterRole, the minimum required, and nothing broader
- For the Vault Agent Injector, use the official Helm chart's RBAC manifests as a baseline rather than custom broad permissions
- Periodically run kubectl auth can-i --list --as=system:serviceaccount:<namespace>:<vault-sa> to verify the effective permission set

Cloud Notes

- AWS: For EKS, also review IAM Roles for Service Accounts (IRSA) annotations on the Vault service account to ensure they do not grant excess AWS IAM permissions in addition to Kubernetes RBAC.
- Azure: For AKS, also review Azure AD Workload Identity bindings on the Vault service account for excess Azure RBAC permissions.

26. Network Policies

Description: Apply Kubernetes NetworkPolicies to restrict which pods and namespaces can communicate with the Vault service, and, if Vault itself runs in-cluster, which pods Vault can reach.

Rationale: Without NetworkPolicies, any pod in the cluster can typically reach the Vault service on its listener port. Restricting ingress to only the namespaces and pods that legitimately need Vault access reduces the attack surface for lateral movement toward Vault.

Impact: Requires a CNI plugin that enforces NetworkPolicies, such as Calico or Cilium. Incomplete policies can inadvertently block legitimate Vault Agent sidecar traffic, causing authentication failures.

Audit

- Run kubectl get networkpolicy -A and confirm policies exist for the Vault namespace restricting ingress to expected client namespaces
- Run kubectl describe networkpolicy <policy> -n <vault-namespace> and review allowed ingress sources and ports, specifically 8200 and 8201
- Verify the cluster's CNI plugin supports NetworkPolicy enforcement

Remediation

- Define a default-deny ingress NetworkPolicy in the Vault namespace, then explicitly allow ingress on port 8200 from namespaces or pods running Vault Agent or applications that call the Vault API directly
- Restrict egress from the Vault namespace to only required destinations, such as the cloud KMS endpoint, Raft peers, and audit log destinations

Cloud Notes

- AWS: For EKS with the VPC CNI, combine Kubernetes NetworkPolicies with Security Groups for Pods for defense in depth.
- Azure: For AKS, ensure Azure CNI network policy, using Calico or the Azure-native option, is enabled at cluster creation, since it cannot be added after the fact without redeploying the cluster.

27. Pod Security

Description: Apply Pod Security Standards, using the restricted or baseline profile via Pod Security Admission, to namespaces running Vault and Vault Agent sidecars, ensuring containers run as non-root with a read-only root filesystem and minimal capabilities where possible.

Rationale: Vault pods handle highly sensitive data in memory, including decrypted secrets and encryption keys. Running with excessive container privileges, such as root, privileged mode, or host namespace access, increases the impact of a container escape or compromised image.

Impact: Some Vault deployment configurations may need targeted exceptions, for example the IPC_LOCK capability required for mlock. The restricted profile may need to be relaxed to baseline with this specific capability allowed.

Audit

- Run kubectl get ns <vault-namespace> -o jsonpath='{.metadata.labels}' and confirm Pod Security Admission labels (pod-security.kubernetes.io/enforce=restricted or baseline) are applied
- Run kubectl get pod <vault-pod> -n <vault-namespace> -o jsonpath='{.spec.securityContext}' and review container-level securityContext for runAsNonRoot, readOnlyRootFilesystem where feasible, and capabilities limited to IPC_LOCK if disable_mlock is not set
- Confirm the cluster uses Pod Security Admission or an equivalent policy engine such as Kyverno or OPA Gatekeeper, since Pod Security Policies were removed in Kubernetes 1.25

Remediation

- Label the Vault namespace with the appropriate Pod Security Admission level: kubectl label ns <vault-namespace> pod-security.kubernetes.io/enforce=restricted

- In the Vault Helm chart values, set securityContext.runAsNonRoot: true, set runAsUser, and configure capabilities to drop ALL and add back only IPC_LOCK if disable_mlock=false
- Use admission controllers such as Gatekeeper or Kyverno to enforce these standards cluster-wide if Pod Security Admission alone is insufficient

Cloud Notes

- AWS: For EKS, combine Pod Security Admission with any AWS-managed admission controls available through EKS add-ons for policy enforcement.
- Azure: For AKS, Azure Policy for AKS can enforce Pod Security Standards cluster-wide as a complement to native Pod Security Admission labels.

28. Secrets Injection

Description: Use the Vault Agent Injector or the Vault CSI Provider to inject secrets into application pods via sidecar or init containers, rendering them as templated files or environment variables, rather than storing secrets in Kubernetes Secret objects or ConfigMaps.

Rationale: Native Kubernetes Secrets are only base64-encoded by default at the API level, unless encryption at rest with envelope encryption is configured, and are often broadly readable via RBAC. The Vault Agent Injector retrieves secrets directly from Vault at pod startup or runtime, scoped by the pod's Kubernetes auth identity, and can automatically renew them.

Impact: Adds a sidecar or init container to each pod, with associated resource overhead, and application startup must tolerate the brief delay while the init container fetches secrets. Debugging requires familiarity with Vault Agent logs in addition to application logs.

Audit

- Run kubectl get pods -n <app-namespace> -o jsonpath='{.items[*].spec.containers[*].name}' and confirm Vault Agent sidecar or init containers are present on pods that require secrets
- Review pod annotations such as vault.hashicorp.com/agent-inject and vault.hashicorp.com/role for correct role and template configuration
- Run kubectl get secrets -n <app-namespace> and confirm sensitive application secrets are not duplicated as plain Kubernetes Secret objects alongside the Vault-injected versions

Remediation

- Deploy the Vault Agent Injector via the official vault-k8s Helm chart
- Annotate application pod specs with vault.hashicorp.com/agent-inject: "true", vault.hashicorp.com/role: <role>, and template annotations specifying which secrets to render and where
- Remove redundant Kubernetes Secret objects once Vault injection is validated, and ensure Kubernetes Secrets encryption at rest using a KMS provider is enabled for any Secrets that remain

Cloud Notes

- AWS: For EKS, ensure encryption at rest for the Kubernetes Secrets API uses an AWS KMS customer managed key via the cluster's encryption configuration, as defense in depth alongside Vault injection.

- Azure: For AKS, confirm etcd encryption at rest is enabled per Azure's documented cluster configuration.

AWS

29. KMS Auto-Unseal

Description: Configure Vault's seal stanza to use AWS KMS for automatic unsealing, using a dedicated customer managed key (CMK).

Rationale: As described in control 4, auto-unseal removes the operational fragility of manual Shamir unseal while delegating protection of the root key to AWS KMS's HSM-backed infrastructure.

Impact: Vault becomes dependent on AWS KMS availability in the configured region. The IAM principal Vault runs as, whether an instance role or an IRSA role, must have access to exactly one CMK, and that access becomes a critical control point.

Audit

- Run vault status and confirm Seal Type: awskms
- Review the server configuration for a seal "awskms" { region = ..., kms_key_id = ... } stanza
- Run aws kms get-key-policy --key-id <id> --policy-name default and confirm the key policy grants encrypt, decrypt, and describe-key only to the Vault IAM role, not to broader principals
- Run aws kms list-grants --key-id <id> and review for unexpected grants

Remediation

- Create a dedicated CMK for Vault auto-unseal: aws kms create-key --description "vault-autounseal"
- Configure seal "awskms" { region = "<region>", kms_key_id = "<key-id>" } in the Vault server configuration
- Restrict the key policy and IAM policy to the specific Vault IAM role and to only the required KMS actions

Cloud Notes

- AWS: Enable CMK rotation with aws kms enable-key-rotation --key-id <id> and enable CloudTrail logging for all KMS API calls against this key.
- Azure: Not applicable; this control is AWS-specific. See control 34 for the Azure equivalent.

30. IAM Auth

Description: Configure Vault's AWS auth method so that EC2 instances, ECS tasks, or Lambda functions can authenticate to Vault using their IAM identity via AWS STS, without static Vault credentials.

Rationale: IAM-based authentication leverages AWS's existing identity infrastructure, which is automatically rotated, eliminating the need to distribute long-lived Vault tokens or credentials to AWS compute resources.

Impact: For EC2, requires IMDSv2 to be correctly configured, per control 33, so the auth method can retrieve the necessary identity information. For ECS and EKS, requires task roles or

IRSA to be correctly scoped, since the IAM role used for Vault auth determines the resulting Vault policy.

Audit

- Run vault auth list and confirm an aws/ mount is enabled
- Run vault read auth/aws/config/client and review the configuration used to validate AWS STS GetCallerIdentity calls
- Run vault list auth/aws/role and vault read auth/aws/role/<role> for each role and confirm bound_iam_principal_arn or equivalent bindings are scoped to specific roles, not account-wide wildcards

Remediation

- Enable the AWS auth method: vault auth enable aws
- Define roles using IAM-type authentication: vault write auth/aws/role/<role> auth_type=iam bound_iam_principal_arn=arn:aws:iam::<account>:role/<app-role> policies=<policy> ttl=1h
- Ensure the EC2, ECS, or Lambda execution role used for the GetCallerIdentity call is itself scoped to least privilege beyond the Vault auth call

Cloud Notes

- AWS: Prefer auth_type=iam over the legacy ec2 auth type, since iam works across EC2, ECS, Lambda, and on-premises-with-AWS-credentials scenarios.
- Azure: Not applicable; see control 35 for the Azure equivalent using Managed Identity.

31. AWS PrivateLink

Description: Where Vault clients need to reach AWS service APIs that Vault depends on, such as KMS for auto-unseal or S3 for Raft snapshot storage, use VPC Endpoints so traffic stays on the AWS private network rather than traversing the public internet.

Rationale: Without VPC endpoints, calls from Vault running in a private subnet to AWS services such as KMS or S3 must route through a NAT Gateway to the public AWS service endpoints, increasing both cost and exposure surface compared with private connectivity.

Impact: VPC endpoints incur their own cost; interface endpoints are billed per hour and per gigabyte, and require corresponding security group and route table configuration. Some AWS services support only Gateway endpoints, such as S3, while others, such as KMS, require Interface endpoints.

Audit

- Run aws ec2 describe-vpc-endpoints --filters Name=vpc-id,Values=<vpc-id> and confirm interface endpoints exist for com.amazonaws.<region>.kms, and gateway or interface endpoints exist for S3 if used for Raft snapshots, in the VPC where Vault runs
- Review route tables for the Vault subnets and confirm traffic to these services routes via the endpoint rather than a NAT Gateway

Remediation

- Create a VPC Interface Endpoint for KMS: aws ec2 create-vpc-endpoint --vpc-id <vpc> --service-name com.amazonaws.<region>.kms --vpc-endpoint-type Interface --subnet-ids <subnets> --security-group-ids <sg>

- Create a Gateway Endpoint for S3 if Vault snapshots are stored there
- Update security groups associated with the endpoints to allow traffic only from Vault's security group

Cloud Notes

- AWS: This control is AWS-specific.
- Azure: Not applicable; see control 36 for the Azure Private Link equivalent.

32. Security Groups

Description: Restrict the security groups attached to Vault instances or nodes so that inbound access to the Vault listener port (8200) and Raft cluster port (8201) is limited to known clients and cluster peers only, not to 0.0.0.0/0.

Rationale: Security groups are the primary network-layer control for AWS-hosted Vault nodes. Overly permissive inbound rules, including rules open to the internet or to entire VPC CIDR ranges, increase exposure to scanning, brute-force, and exploitation attempts against the Vault API.

Impact: Overly restrictive security groups can break legitimate client access or inter-node Raft communication, requiring careful enumeration of all legitimate source security groups or CIDRs, including load balancers, application subnets, monitoring tools, and other Vault nodes.

Audit

- Run aws ec2 describe-security-groups --group-ids <sg-id> and review inbound rules for ports 8200 and 8201, flagging any rule with a source of 0.0.0.0/0 or ::/0
- Confirm port 8201, used for Raft and cluster communication, is restricted to the security group of other Vault nodes only and is not exposed to application subnets
- Confirm port 8200 is restricted to the load balancer's security group or specific client subnets, not the entire VPC

Remediation

- Remove any 0.0.0.0/0 or ::/0 ingress rules on ports 8200 and 8201
- Scope port 8200 ingress to the load balancer's security group or specific client security groups only
- Scope port 8201 ingress to a security group containing only Vault cluster nodes

Cloud Notes

- AWS: This control is AWS-specific.
- Azure: See control 37 for the Network Security Group equivalent.

33. IMDSv2 Enforcement

Description: Ensure all EC2 instances running Vault, and instances using the AWS auth method to authenticate to Vault, enforce Instance Metadata Service Version 2 with HttpTokens set to required.

Rationale: IMDSv1 is vulnerable to SSRF-based credential theft, where an attacker exploiting a server-side request forgery vulnerability in an application can retrieve the instance's IAM role credentials from the metadata service. Since Vault and AWS auth method clients rely on instance IAM roles for KMS auto-unseal and AWS authentication, protecting metadata access is critical.

Impact: Applications or agents with hardcoded IMDSv1-only metadata calls will fail and need updating to use the IMDSv2 token-based flow; this is a one-time remediation for legacy tooling.

Audit

- Run aws ec2 describe-instances --instance-ids <id> --query 'Reservations[].Instances[].MetadataOptions' and confirm HttpTokens: required and HttpEndpoint: enabled, with HttpPutResponseHopLimit set appropriately for containerized workloads
- For Auto Scaling Groups, check the launch template's MetadataOptions to confirm new instances inherit IMDSv2 enforcement

Remediation

- Update existing instances: aws ec2 modify-instance-metadata-options --instance-id <id> --http-tokens required --http-endpoint enabled
- Update launch templates and Auto Scaling Group configurations to set MetadataOptions.HttpTokens=required for all new instances
- For containerized workloads on EC2 that need to reach the metadata service through a NAT or proxy, set an appropriate HttpPutResponseHopLimit, for example 2

Cloud Notes

- AWS: This control is AWS-specific.
- Azure: Not directly applicable; Azure's equivalent Instance Metadata Service does not have an analogous version distinction, but access to it should similarly be restricted via NSGs and host firewall rules.

Azure

34. Key Vault Auto-Unseal

Description: Configure Vault's seal stanza to use Azure Key Vault for automatic unsealing, using a dedicated key with a restricted access policy or RBAC assignment.

Rationale: As with AWS KMS auto-unseal in control 29, this removes the operational fragility of manual Shamir unseal while delegating protection of the root key to Azure Key Vault's HSM-backed infrastructure on the Premium tier, or software-protected keys on the Standard tier.

Impact: Vault becomes dependent on Azure Key Vault availability. The Managed Identity or Service Principal Vault runs as must have wrapKey and unwrapKey permissions on exactly one key, and Key Vault soft-delete and purge protection should be enabled to prevent accidental key loss.

Audit

- Run vault status and confirm Seal Type: azurekeyvault
- Review the server configuration for a seal "azurekeyvault" { vault_name = ..., key_name = ... } stanza
- Run az keyvault key show --vault-name <vault> --name <key> and az keyvault show --name <vault> --query "properties.enablePurgeProtection" and confirm purge protection and soft-delete are enabled
- Review role assignments or access policies on the Key Vault and confirm only the Vault Managed Identity has wrapKey, unwrapKey, and get permissions on this key

Remediation

- Create a dedicated Key Vault key for auto-unseal: az keyvault key create --vault-name <vault> --name vault-autounseal
- Configure seal "azurekeyvault" { vault_name = "<vault>", key_name = "vault-autounseal", ... } in the Vault server configuration
- Enable soft-delete and purge protection on the Key Vault, and restrict access via RBAC to the Vault Managed Identity only

Cloud Notes

- AWS: Not applicable; this control is Azure-specific. See control 29 for the AWS equivalent.
- Azure: Azure Key Vault Premium uses HSM-backed keys validated at FIPS 140-3 Level 3 for all keys created on HSM Platform 2 (the current default for new HSM keys). Keys created on the older HSM Platform 1 were validated at FIPS 140-2 Level 2. If your compliance requirement explicitly mandates dedicated, single-tenant FIPS 140-3 Level 3 hardware, use Azure Managed HSM rather than standard Key Vault Premium, as Azure Managed HSM provides that isolation guarantee. Confirm the HSM platform generation for your specific key before making compliance attestations.

35. Managed Identity

Description: Configure Vault's Azure auth method so that Azure VMs, Virtual Machine Scale Sets, or AKS pods using Azure AD Workload Identity can authenticate to Vault using their assigned Managed Identity, without static credentials.

Rationale: Managed Identity-based authentication leverages Azure AD-issued, automatically rotated tokens tied to the resource's identity, eliminating the need to distribute long-lived Vault credentials to Azure compute resources, directly analogous to AWS IAM auth in control 30.

Impact: Requires the Azure auth method to be configured with the correct tenant, resource group, and subscription bounds. For AKS, requires Azure AD Workload Identity, or the deprecated AAD Pod Identity, to be correctly configured so pods receive Managed Identity tokens.

Audit

- Run vault auth list and confirm an azure/ mount is enabled
- Run vault read auth/azure/config and confirm tenant_id, resource, and any client credentials are correctly scoped, preferring Vault running with a Managed Identity over a static client secret where possible
- Run vault list auth/azure/role and vault read auth/azure/role/<role> for each role and confirm bound_subscription_ids, bound_resource_groups, and bound_service_principal_ids are scoped to specific resources rather than subscription-wide wildcards where avoidable

Remediation

- Enable the Azure auth method: vault auth enable azure
- Configure tenant and resource information: vault write auth/azure/config tenant_id=<tenant> resource=https://management.azure.com/ ...
- Define roles scoped to specific resource groups or identities: vault write auth/azure/role/<role> bound_subscription_ids=<subscription> bound_resource_groups=<resource-group> policies=<policy> ttl=1h

Cloud Notes

- AWS: Not applicable; see control 30 for the AWS equivalent.
- Azure: For AKS workloads, prefer Azure AD Workload Identity using federated credentials over the deprecated AAD Pod Identity.

36. Azure Private Link

Description: Where Vault depends on Azure service APIs, such as Key Vault for auto-unseal or a Storage Account for Raft snapshots, use Private Endpoints so traffic stays on the Azure backbone rather than traversing the public internet.

Rationale: Without Private Endpoints, calls from Vault to Azure Key Vault or Storage Accounts traverse Azure's public service endpoints even when originating from a private virtual network, increasing the exposure surface relative to fully private connectivity, directly analogous to AWS PrivateLink in control 31.

Impact: Private Endpoints require corresponding Private DNS Zone configuration so that the service's standard hostname resolves to the private IP address; without correct DNS, clients may continue to reach the public endpoint.

Audit

- Run az network private-endpoint list --resource-group <resource-group> and confirm private endpoints exist for the Key Vault and Storage Account used by Vault
- Run az network private-dns zone list and review zone records to confirm the Private DNS Zone for privatelink.vaultcore.azure.net (Key Vault) and privatelink.blob.core.windows.net (Storage), as applicable, are linked to the virtual network with correct records
- Run az keyvault show --query "properties.publicNetworkAccess" and confirm public network access is disabled on the Key Vault and Storage Account once private endpoints are validated

Remediation

- Create a Private Endpoint for the Key Vault: az network private-endpoint create --resource-group <resource-group> --name <pe-name> --vnet-name <vnet> --subnet <subnet> --private-connection-resource-id <keyvault-id> --group-id vault --connection-name <connection-name>
- Create and link a corresponding Private DNS Zone, and disable public network access on the resource once the private endpoint is validated

Cloud Notes

- AWS: Not applicable; see control 31 for the AWS equivalent.
- Azure: This control is Azure-specific.

37. NSGs

Description: Restrict Network Security Groups on subnets and network interfaces hosting Vault so that inbound access to the Vault listener port (8200) and Raft cluster port (8201) is limited to known clients and cluster peers only.

Rationale: NSGs are the primary network-layer control for Azure-hosted Vault nodes, directly analogous to AWS Security Groups in control 32. Overly permissive rules, such as a source of Any or Internet, increase exposure to scanning and exploitation attempts.

Impact: Overly restrictive NSGs can break legitimate client access or inter-node Raft communication, requiring careful enumeration of legitimate source NSGs or subnets, including Application Gateway, application subnets, other Vault nodes, and monitoring.

Audit

- Run az network nsg rule list --resource-group <resource-group> --nsg-name <nsg> and review inbound rules for ports 8200 and 8201, flagging any rule with a source of *, Internet, or 0.0.0.0/0
- Confirm port 8201 is restricted to an Application Security Group or subnet containing only Vault cluster nodes
- Confirm port 8200 is restricted to the Application Gateway or Load Balancer subnet, or to specific client subnets

Remediation

- Remove any rules permitting Internet or Any as the source on ports 8200 and 8201
- Scope port 8200 ingress to the Application Gateway subnet or Application Security Group
- Scope port 8201 ingress to an Application Security Group containing only Vault cluster nodes

Cloud Notes

- AWS: See control 32 for the Security Group equivalent.
- Azure: This control is Azure-specific.

38. AAD Integration

Description: Integrate Vault authentication, or the Azure secrets engine, with Azure AD (Microsoft Entra ID), either via the OIDC auth method described in control 6 for human users, or via the Azure secrets engine for generating short-lived Service Principal credentials.

Rationale: Azure AD integration centralizes identity governance, including conditional access, MFA, and group management, for both human Vault users and Azure-targeted dynamic secrets, ensuring Vault access and Azure resource access follow the same identity lifecycle.

Impact: Requires an App Registration in Azure AD with appropriate API permissions and admin consent for the OIDC scopes. For the Azure secrets engine, requires Vault to hold a Service Principal with sufficient privilege to create and delete short-lived Service Principals, itself a high-value credential requiring strict access controls.

Audit

- Run vault read auth/oidc/config and confirm oidc_discovery_url points to the correct Azure AD tenant, in the form https://login.microsoftonline.com/<tenant-id>/v2.0
- Run vault read azure/config for the Azure secrets engine and confirm tenant_id, client_id, and the scoped subscription are correct
- Run az ad app list --display-name <vault-app-name> and review the API permissions granted, confirming only the required Microsoft Graph or Azure Management permissions are granted with admin consent tracked

Remediation

- Register an App Registration for Vault in Azure AD, request only the minimum required API permissions, and obtain admin consent
- Configure vault write auth/oidc/config oidc_discovery_url=https://login.microsoftonline.com/<tenant-id>/v2.0 oidc_client_id=<app-id> oidc_client_secret=<secret>
- For the Azure secrets engine, create a dedicated Service Principal for Vault scoped to the minimum required application management permissions rather than a broad Application Administrator role

Cloud Notes

- AWS: Not applicable; see control 6 for the general identity federation pattern.
- Azure: This control is Azure-specific.

Monitoring

39. Enable Metrics

Description: Enable Vault's telemetry stanza to expose operational metrics, including request rates, latency, lease counts, token counts, and storage backend health, via Prometheus, StatsD, or another supported sink.

Rationale: Metrics are essential for detecting operational issues, such as unsealing failures, storage backend latency, or lease churn indicating credential abuse, before they become outages, and for capacity planning.

Impact: Exposing a Prometheus metrics endpoint requires its own access controls, since it can reveal operational details about Vault's internal state, and should not be exposed without authentication or network restriction.

Audit

- Review the server configuration for a telemetry { ... } stanza
- Query the metrics endpoint with an appropriate token, for example curl -H "X-Vault-Token: <token>" <vault-addr>/v1/sys/metrics?format=prometheus, and confirm it returns data
- Confirm the metrics endpoint or port is not exposed beyond the monitoring system's network segment

Remediation

- Add a telemetry stanza to the server configuration, for example telemetry { prometheus_retention_time = "24h", disable_hostname = true }
- Restrict access to sys/metrics via policy, requiring a token with the metrics capability, and through network controls
- Configure a Prometheus scrape job, or a StatsD or Telegraf agent, to collect Vault metrics

Cloud Notes

- AWS: Forward metrics to Amazon Managed Service for Prometheus, or to CloudWatch via a metrics agent, for centralized dashboards and alerting.
- Azure: Forward metrics to Azure Monitor managed Prometheus or Azure Managed Grafana.

40. Central Logging

Description: Forward Vault server logs and audit logs, as described in control 3, to a centralized logging platform separate from the host filesystem, with appropriate retention.

Rationale: Centralized logging ensures logs survive host failure or compromise, since an attacker who compromises a Vault host could otherwise tamper with or delete local logs to cover their tracks, and enables correlation with logs from other systems during an investigation.

Impact: Requires a reliable log shipping pipeline. If the central logging destination becomes unavailable, local buffering and retention must be sufficient to avoid log loss; per control 3, audit log write failures will block all Vault requests if the configured audit device itself becomes unreachable.

Audit

- Confirm a log shipping agent, such as Fluent Bit, Filebeat, or rsyslog, is running on Vault hosts and configured to forward both server logs and audit logs
- Verify recent log entries are visible in the central logging platform with correct timestamps and no significant lag or gaps
- Confirm local log retention provides a sufficient buffer, for example 24 to 72 hours, in case of shipping interruptions

Remediation

- Deploy a log shipping agent on all Vault hosts or pods, configured to tail both the server log and audit log files
- Configure the central logging platform with retention aligned to compliance requirements, often one year or more for security audit logs
- Set up basic log-volume alerting to detect shipping pipeline failures

Cloud Notes

- AWS: Ship logs to CloudWatch Logs, with subscription filters to S3 or OpenSearch for long-term retention and search, or directly to an S3 bucket with Object Lock enabled for immutability.
- Azure: Ship logs to a Log Analytics workspace via the Azure Monitor Agent, with Diagnostic Settings configured for retention.

41. SIEM Integration

Description: Integrate Vault audit logs and operational alerts with the organization's SIEM, such as Splunk, Sentinel, QRadar, or Elastic Security, for correlation, threat detection, and compliance reporting.

Rationale: Vault audit logs contain detailed records of every authentication and secret access, but raw logs are only useful if actively monitored. SIEM integration enables detection rules for suspicious patterns, such as a token suddenly accessing many unrelated secret paths, repeated authentication failures, or root token usage, and supports compliance evidence collection.

Impact: Requires log format parsing and normalization, since Vault audit logs are JSON with hashed sensitive values by default via HMAC-SHA256. SIEM ingestion volume from a busy Vault cluster can be significant and should be accounted for in licensing and sizing.

Audit

- Confirm Vault audit log events are visible and correctly parsed in the SIEM, checking field extraction for fields such as type, auth.policies, request.path, and response fields
- Review existing SIEM detection rules and use cases for Vault-specific scenarios, such as root token use, policy changes, and mass secret reads
- Confirm log delivery latency from Vault to the SIEM meets the organization's detection SLA

Remediation

- Configure the central logging pipeline from control 40 to forward audit logs to the SIEM, with appropriate JSON parsing and field mapping
- Implement detection rules for, at minimum, root token usage, modifications to sys/policy or sys/policies, authentication failures exceeding a threshold, and unusual volumes of reads under secret/data/* by a single token or entity

- Periodically review the audit device's HMAC salt configuration if log correlation across time periods is required

Cloud Notes

- AWS: If CloudWatch Logs is used as an intermediate store, use a subscription filter or a Lambda forwarder to deliver logs to the SIEM.
- Azure: If a Log Analytics workspace is used, integrate natively with Sentinel if that is the SIEM, or configure Event Hub export for third-party SIEMs.

42. Alerting Rules

Description: Define alerting rules on Vault's operational metrics from control 39 and audit and log events from controls 40 and 41 for conditions requiring immediate attention, including seal status changes, leader election or failover, high latency, certificate expiry, and security-relevant audit events.

Rationale: Metrics and logs only provide value if someone is notified when something goes wrong. Without alerting, a sealed Vault cluster, a failing audit device that blocks all Vault requests, or a security incident may go unnoticed until users report an outage.

Impact: Poorly tuned alerts that are too sensitive lead to alert fatigue and ignored notifications, while thresholds that are too lenient delay response to genuine incidents. Alert thresholds should be reviewed periodically based on observed baseline behavior.

Audit

- Review the alerting and monitoring platform for configured rules covering, at minimum, unsealed status changes, leadership transitions, audit device write failures, certificate expiry on the Vault TLS listener and PKI-issued certificates, and Raft cluster health
- Confirm alert routing, for example to PagerDuty, Opsgenie, email, or Slack, is tested and reaches the appropriate on-call team

Remediation

- Implement alerts for Vault becoming sealed unexpectedly, leader elections occurring outside planned maintenance, the Raft peer count falling below quorum, audit device errors, TLS or PKI certificate expiry within a defined number of days, and elevated error rates in sys/metrics
- Route security-relevant alerts, such as root token usage or repeated authentication failures, separately to the security team via the SIEM from control 41, distinct from operational alerts to the platform team

Cloud Notes

- AWS: Implement alerting via CloudWatch Alarms for metrics shipped to CloudWatch, with SNS notification routing.
- Azure: Implement alerting via Azure Monitor Alerts for metrics in Azure Monitor or Log Analytics, with Action Groups for notification routing.

DR

43. Encrypted Backups

Description: Ensure Raft snapshots, or storage backend backups, are protected by encryption at rest at the storage location, in addition to the encryption Vault itself applies to its stored data.

Rationale: A Raft snapshot contains an encrypted copy of Vault's storage, since Vault encrypts data at rest with its own barrier key, so snapshot contents are themselves encrypted. The storage location for snapshots, such as an S3 bucket or Storage Account, should still enforce encryption at rest and access controls as defense in depth, since snapshot files are a concentrated, portable copy of the entire Vault dataset.

Impact: Restoring an encrypted snapshot requires the Vault cluster performing the restore to have access to the same auto-unseal mechanism, such as the same KMS or Key Vault key, used by the original cluster, or to the original Shamir keys. This dependency must be documented as part of the DR plan.

Audit

- Confirm the storage location for vault operator raft snapshot save output, such as an S3 bucket or Storage Account, has encryption at rest enabled
- Confirm access controls on the snapshot storage location restrict access to authorized backup and DR roles only
- Confirm DR documentation specifies which KMS key, Key Vault key, or Shamir key shares are required to unseal a restored cluster

Remediation

- Enable default encryption on the snapshot storage location, for example SSE-KMS with a dedicated customer managed key on AWS, or customer-managed key encryption on an Azure Storage Account
- Restrict IAM or RBAC permissions on the snapshot storage location to a dedicated backup role, separate from general Vault administrator access
- Document the auto-unseal key dependency for snapshot restoration in the DR runbook

Cloud Notes

- AWS: Store snapshots in S3 with SSE-KMS, using a CMK distinct from the auto-unseal CMK or the same CMK with appropriate cross-account grants for DR, and enable S3 Versioning and Object Lock.
- Azure: Store snapshots in a Storage Account with customer-managed key encryption and immutable blob storage policies enabled.

44. Snapshot Protection

Description: Protect stored Raft snapshots from unauthorized access, modification, or deletion, including enabling versioning or immutability and restricting the set of identities able to read or delete snapshot files.

Rationale: A Raft snapshot is effectively a portable copy of the entire Vault dataset, encrypted but still complete. If an attacker can read it and also obtain the auto-unseal key, or can delete or tamper with it, this directly threatens both confidentiality, via offline attack on a stolen snapshot, and recoverability, via deletion ahead of a destructive attack on the live cluster.

Impact: Immutability or Object Lock policies prevent legitimate deletion of old snapshots until retention periods expire, which must be factored into storage cost and lifecycle planning.

Audit

- On AWS, run aws s3api get-bucket-versioning --bucket <bucket> and aws s3api get-object-lock-configuration --bucket <bucket> and confirm versioning and Object Lock are enabled on the snapshot bucket
- On Azure, run az storage account blob-service-properties show and confirm immutability policies are configured on the snapshot container
- Review IAM or RBAC policies for the snapshot storage location and confirm only the backup automation principal and a small DR admin group have read access, with delete permissions even more restricted

Remediation

- Enable S3 Versioning and Object Lock, in compliance or governance mode per organizational policy, on the snapshot bucket, or enable immutable blob storage policies on Azure
- Restrict snapshot storage access via IAM policy or RBAC to the backup automation role, write-only where possible, and a separate DR admin role for restore testing
- Enable access logging on the snapshot storage location itself

Cloud Notes

- AWS: Combine S3 Object Lock with cross-account replication to a separate AWS account dedicated to backups, for resilience against ransomware or insider threat scenarios.
- Azure: Combine immutable blob storage with cross-region replication, such as GRS or RA-GRS, for resilience.

45. Restore Testing

Description: Periodically perform a full restore of a Raft snapshot into a non-production environment to validate that backups are usable and that the restore procedure, including auto-unseal, works as documented.

Rationale: Backups that have never been tested for restoration are not a reliable DR control. Issues such as corrupted snapshots, missing unseal or recovery keys, or undocumented manual steps are commonly discovered only during an actual incident if not tested in advance.

Impact: Restore testing requires a dedicated non-production Vault environment to avoid impacting production, and consumes time and resources on a recurring schedule, commonly quarterly or after any change in Vault version or topology.

Audit

- Review DR runbook and test logs for evidence of a recent successful restore test, within the last 6 to 12 months, using vault operator raft snapshot restore
- Confirm the restore test included validation of auto-unseal, or Shamir unseal using documented key material, and a basic application-level smoke test against the restored cluster

Remediation

- Schedule recurring restore tests, for example quarterly, in an isolated environment using the most recent production snapshot
- Document the full restore procedure step by step, including unseal or recovery key retrieval, and update it whenever Vault version or topology changes
- Track restore test results, including success or failure and time-to-restore, as a DR metric

Cloud Notes

- AWS: Use a separate AWS account or VPC for the DR test environment to ensure isolation from production.
- Azure: Use a separate subscription or virtual network for the DR test environment.

46. Unseal and Recovery Key Security

Description: For clusters using auto-unseal as described in controls 4, 29, and 34, securely store and control access to the recovery key shares. For clusters still using Shamir unseal, ensure unseal key shares are distributed to separate individuals and stored in separate secure locations so that no single person or location holds a quorum.

Rationale: Recovery keys, when auto-unseal is used, are required for operations such as generate-root, rekey, and certain DR operations, and therefore retain significant power even though they are not needed for routine unsealing. Shamir unseal keys, when auto-unseal is not used, defeat the purpose of key splitting if held by a single person or stored together, since any one person or location compromise could reconstruct the master key.

Impact: Recovery or unseal key custodians must be available when operations requiring them arise, such as an emergency generate-root. If too many custodians leave the organization without a documented succession process, quorum may become unobtainable.

Audit

- Run vault status and confirm whether the cluster uses auto-unseal, indicated by a Recovery Seal Type, rather than relying on Shamir unseal key shares for routine operation
- For auto-unseal clusters, review documentation and process for where recovery key shares are stored and who holds them, confirming no single person holds a quorum, for example a 3-of-5 share configuration held by 5 distinct individuals or locations
- For clusters without auto-unseal, apply the same review to unseal key shares directly
- Confirm key custodians are current employees with documented succession and handover procedures

Remediation

- If auto-unseal is not already in use, implement controls 4, 29, or 34 to reduce day-to-day reliance on Shamir unseal keys

- Distribute recovery or unseal key shares to a quorum, for example 5 shares with a threshold of 3, of distinct individuals across distinct secure storage locations, such as physical safes in different offices or separate password managers and HSM-backed vaults
- Run vault operator rekey if the key share distribution needs to change, for example after a custodian departs, and document the updated custodian list

Cloud Notes

- AWS: Do not store all recovery key shares in a single AWS Secrets Manager secret or account; distribute across separate accounts or off-platform secure storage.
- Azure: Do not store all recovery key shares in a single Key Vault; distribute across separate subscriptions or off-platform secure storage.

Reference Architectures

The following reference architectures illustrate how the controls in this benchmark combine in common deployment patterns. Control numbers in parentheses refer to the corresponding sections above.

Vault HA (Raft)

Multiple nodes, typically 3 or 5, are deployed behind a load balancer with TLS enabled on all listeners (control 1) and auto-unseal configured via a cloud KMS (control 4). Nodes are distributed across availability zones for resilience, consistent with control 2.

Vault + EKS

Vault is deployed via the official Helm chart and integrated with the Kubernetes auth method (control 24) for workload authentication. IAM Roles for Service Accounts (IRSA) are used to grant Vault scoped access to AWS APIs such as KMS, following the least-privilege guidance in controls 9 and 29.

Vault + AKS

Vault runs with Azure AD Workload Identity / Managed Identity for Azure API access (control 35), and Private Link is used for connectivity to Azure Key Vault and Storage Accounts (control 36), avoiding traversal of public endpoints.

Network Segmentation

Vault is isolated in private subnets or a dedicated VNet/subnet, with strict ingress and egress rules enforced via Security Groups (control 32) on AWS or Network Security Groups (control 37) on Azure, and no direct internet exposure to the Vault listener or Raft cluster ports.